

ASSIGNMENT 04

Kinza 23K-0842

December 7, 2025

Question 1

Consider the following scenario: A ride-hailing app processes a transaction where a customer books a ride, the driver accepts it, and payment is reserved from the wallet. During this process:

- The driver accepted the ride (system recorded).
- The customer's wallet balance was deducted.
- But before the ride confirmation was shown to the customer, the server crashed.

(a) Violated ACID Property/Properties

In this scenario, the ACID properties violated are:

- **Atomicity:** Not all parts of the transaction completed successfully. Although the driver accepted the ride and wallet was deducted, the customer did not receive confirmation. Partial changes exist in the system.
- **Durability:** Since the server crashed before confirmation was saved, there is uncertainty about whether the recorded actions (driver acceptance, wallet deduction) are permanently stored.

(b) Reason for Violation

- **Atomicity Violation:** The transaction involved multiple steps: recording driver acceptance, deducting wallet balance, and confirming the ride. A failure occurred after some steps were executed but before the transaction committed, leaving the system in an inconsistent state.
- **Durability Risk:** The crash may prevent the database from persisting changes to disk, depending on the transaction commit strategy. Without proper logging or commit protocols, data can be lost.

(c) Revised Scenario Preserving ACID Properties

To ensure ACID compliance, the transaction should be structured as an atomic unit:

1. Begin a database transaction.
2. Record the driver's acceptance of the ride.
3. Reserve (or deduct) the customer's wallet balance.
4. Update the ride status to "confirmed".
5. Commit the transaction.

```
CREATE OR REPLACE PROCEDURE book_ride(  
    IN p_ride_id INT,  
    IN p_customer_id INT,  
    IN p_driver_id INT,  
    IN p_amount INT  
)  
AS  
BEGIN  
    -- Begin Transaction  
    BEGIN  
  
        -- Step 1: Driver accepts the ride  
        UPDATE Rides  
        SET driver_status = 'accepted',
```

```

        driver_id = p_driver_id
WHERE ride_id = p_ride_id;

-- Step 2: Deduct or reserve customer's wallet balance
UPDATE Wallets
SET balance = balance - p_amount
WHERE customer_id = p_customer_id;

-- Step 3: Mark ride as confirmed
UPDATE Rides
SET ride_status = 'confirmed'
WHERE ride_id = p_ride_id;

-- Commit if all steps succeed
COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        -- Rollback on any error or crash
        ROLLBACK;
        RAISE NOTICE 'Transaction failed. All changes reverted.';
END;
END;
```

Key Points:

- If the server crashes before commit, the transaction is rolled back; no partial updates remain.
- Once committed, all changes are durable, and the customer sees confirmation.
- Transaction logging ensures recovery after a crash.

(d) Long-Term Business Consequences of Frequent Violations

- **Customer dissatisfaction:** Customers may be charged without ride confirmation, eroding trust.

- **Driver frustration:** Drivers may see rides accepted without actual execution, causing confusion.
- **Financial losses:** Reconciliation issues and refunds may increase operational costs.
- **Reputation damage:** Persistent transaction failures may reduce platform reliability and lead to user churn.

Question 2

Two transactions run concurrently on an online inventory system:

$$\begin{aligned} T1: & \begin{cases} \text{Read}(P1.quantity) \\ \text{Write}(P1.quantity - sold_units) \end{cases} \\ T2: & \begin{cases} \text{Read}(P1.quantity) \\ \text{Write}(P1.quantity + returned_units) \end{cases} \end{aligned}$$

Since both run together, the final quantity becomes inconsistent.

(a) Non-Serial Schedule

A possible non-serial (interleaved) schedule is:

$$S = R_1(P1.quantity), R_2(P1.quantity), W_1(P1.quantity), W_2(P1.quantity)$$

This means:

- T1 reads P1.quantity
- T2 reads P1.quantity
- T1 writes the new quantity after sale
- T2 writes the new quantity after return

(b) Precedence Graph Construction

We build the conflict graph using conflicting operations:

- Read–Write Conflict
- Write–Read Conflict
- Write–Write Conflict

Conflicts identified:

1. $R_1(P1)$ and $W_2(P1) \rightarrow \text{conflict} \rightarrow \text{edge } T1 \rightarrow T2$
2. $R_2(P1)$ and $W_1(P1) \rightarrow \text{conflict} \rightarrow \text{edge } T2 \rightarrow T1$
3. $W_1(P1)$ and $W_2(P1) \rightarrow \text{conflict (write–write)}$

Thus, the precedence graph edges are:

$$T1 \rightarrow T2, \quad T2 \rightarrow T1$$

This produces a cycle.

$$T1 \rightarrow T2 \rightarrow T1$$

(c) Conflict Serializability

A schedule is conflict-serializable *if and only if* its precedence graph has no cycles.

Since the graph contains a cycle:

$$T1 \rightarrow T2 \rightarrow T1$$

The schedule is NOT conflict-serializable.

(d) Corrected Serializable Schedule

A serial schedule that ensures consistency:

$$S' = R_1(P1), W_1(P1), R_2(P1), W_2(P1)$$

This is equivalent to executing:

$$T1 \rightarrow T2$$

Or the opposite serial order:

$$S'' = R_2(P1), W_2(P1), R_1(P1), W_1(P1)$$

Both are conflict-serializable and avoid inconsistent inventory updates.

(e) Real-World Business Loss

If such anomalies occur frequently, the company may face:

- **Incorrect inventory counts:** Over-selling or under-stocking.
- **Lost revenue:** Selling products that are no longer available.
- **Customer dissatisfaction:** Cancelled orders or shipment delays.
- **Warehouse inefficiency:** Wrong reorder decisions.
- **Financial loss:** Misreporting stock leads to inaccurate accounting.

Question 3

Transactions:

- $T1 : r_1(a); r_1(b); w_1(a)$
- $T2 : r_2(b); r_2(c); w_2(b)$
- $T3 : r_3(c); r_3(d); w_3(c)$
- $T4 : r_4(d); r_4(a); w_4(d)$

Schedules:

- $S1 : r_1(a), r_2(b), r_3(c), r_4(d), r_1(b), r_2(c), r_3(d), r_4(a), w_2(b), w_3(c), w_4(d), w_1(a)$
- $S2 : r_4(d), r_4(a), w_4(d), r_3(d), r_3(c), w_3(c), r_2(b), r_2(c), w_2(b), r_1(a), r_1(b), w_1(a)$

(1) Precedence Graph for S1

We inspect conflicts (read–write, write–read, write–write on same item between different transactions) in the order given by $S1$.

Conflicting edges found in $S1$:

- On item b : $r_1(b)$ (by $T1$ at position 5) precedes $w_2(b)$ (by $T2$ at position 9) $\Rightarrow T1 \rightarrow T2$.
- On item c : $r_2(c)$ (by $T2$ at pos 6) precedes $w_3(c)$ (by $T3$ at pos 10) $\Rightarrow T2 \rightarrow T3$.
- On item d : $r_3(d)$ (by $T3$ at pos 7) precedes $w_4(d)$ (by $T4$ at pos 11) $\Rightarrow T3 \rightarrow T4$.
- On item a : $r_4(a)$ (by $T4$ at pos 8) precedes $w_1(a)$ (by $T1$ at pos 12) $\Rightarrow T4 \rightarrow T1$.

Thus the precedence graph for $S1$ has edges:

$$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T1$$

which forms a cycle. Figure 2 shows the precedence graph.

(2) Serializability of S1

Because the precedence graph contains the cycle

$$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T1,$$

$S1$ is **not conflict-serializable**.

(1) Precedence Graph for S2

Examine conflicts in the order of $S2$. Conflicts found:

- On item d : $w_4(d)$ (by $T4$ at pos 3) precedes $r_3(d)$ (by $T3$ at pos 4)
 $\Rightarrow T4 \rightarrow T3$.
- On item c : $w_3(c)$ (by $T3$ at pos 6) precedes $r_2(c)$ (by $T2$ at pos 8)
 $\Rightarrow T3 \rightarrow T2$.
- On item b : $w_2(b)$ (by $T2$ at pos 9) precedes $r_1(b)$ (by $T1$ at pos 11)
 $\Rightarrow T2 \rightarrow T1$.
- On item a : $r_4(a)$ (by $T4$ at pos 2) precedes $w_1(a)$ (by $T1$ at pos 12)
 $\Rightarrow T4 \rightarrow T1$.

Edges collected:

$$T4 \rightarrow T3 \rightarrow T2 \rightarrow T1 \quad \text{and} \quad T4 \rightarrow T1$$

(no edge leads back to $T4$).

TikZ drawing:

(2) Serializability of S2

The precedence graph for $S2$ is **acyclic**. Therefore $S2$ is **conflict-serializable**.

(3) Equivalent Serial Schedule(s) for S2

Topological orders consistent with the graph give equivalent serial schedules.
One valid topological order is:

$$T4 \rightarrow T3 \rightarrow T2 \rightarrow T1.$$

So $S2$ is conflict-equivalent to the serial schedule executing $T4$, then $T3$, then $T2$, then $T1$ (this order respects all directed edges).

Summary:

- $S1$: precedence graph has a cycle $\Rightarrow$ *not* conflict-serializable.
- $S2$: precedence graph is acyclic $\Rightarrow$ conflict-serializable; equivalent serial schedule $T4, T3, T2, T1$.

Question 4

Given: Table `Cart`, for `User_ID = 101` the initial `Total_Amount = Rs 2000`.

Two concurrent transactions:

$T1$: apply coupon (-300)

$T2$: add product ($+800$)

Interleaved execution (given):

1. $T1$ reads $TA = 2000$
2. $T2$ reads $TA = 2000$
3. $T1$ computes $2000 - 300 = 1700$
4. $T2$ computes $2000 + 800 = 2800$
5. $T1$ writes $TA \leftarrow 1700$
6. $T2$ writes $TA \leftarrow 2800$

(1) Concurrency problem(s) and how they arise

Problems present

- **Lost update (write–write conflict):** One transaction's update is overwritten by the other's write because both read the same initial value and then write computed results based on that stale value.
- **Race condition / non-serializable schedule:** The operations are interleaved so that the outcome does not match any serial execution of the two transactions.

How they arise: Both $T1$ and $T2$ read the same initial value (2000) before either has written. Each computes a new value independently and writes it back. Because the second writer overwrites the first writer's result, one update is effectively lost. This happens when transactions do *read-modify-write* without concurrency control (locks, atomic DB updates, or higher isolation). The root cause is the absence of mutual exclusion on the shared data item (`Total_Amount`).

(2) Final Total_Amount after the given interleaved schedule

Following the last write in the interleaving (step 6), the stored value is:

$$\boxed{\text{Total_Amount} = \text{Rs } 2800.}$$

(Reason: T_2 's write of 2800 overwrote T_1 's prior write of 1700.)

(3) Correct final Total_Amount when executed serially

Compute both serial orders:

Order 1: $T_1 \rightarrow T_2$ (coupon first, then add product)

$$2000 - 300 = 1700 \quad (\text{after } T_1) \quad 1700 + 800 = 2500 \quad (\text{after } T_2)$$

Final amount: $\boxed{\text{Rs } 2500.}$

Order 2: $T_2 \rightarrow T_1$ (add product first, then coupon)

$$2000 + 800 = 2800 \quad (\text{after } T_2) \quad 2800 - 300 = 2500 \quad (\text{after } T_1)$$

Final amount: $\boxed{\text{Rs } 2500.}$

Conclusion Both serial orders produce the same correct final value:

$$\boxed{\text{Correct Total_Amount} = \text{Rs } 2500.}$$

The interleaved schedule produced an incorrect result (Rs 2800) due to a lost update.

Question 5

Determine which schedules are conflict-serializable. For each serializable schedule give the equivalent serial schedule(s).

(a) $S_a : r_1(X); w_1(X); r_3(X); r_2(X); w_3(X).$

Conflicts (in execution order):

- $w_1(X)$ (pos 2) before $r_3(X)$ (pos 3) $\Rightarrow T1 \rightarrow T3$ (WR).
- $w_1(X)$ (pos 2) before $r_2(X)$ (pos 4) $\Rightarrow T1 \rightarrow T2$ (WR).
- $r_2(X)$ (pos 4) before $w_3(X)$ (pos 5) $\Rightarrow T2 \rightarrow T3$ (RW).
- $w_1(X)$ (pos 2) before $w_3(X)$ (pos 5) $\Rightarrow T1 \rightarrow T3$ (WW) (already present).

Precedence graph edges: $T1 \rightarrow T2$, $T1 \rightarrow T3$, $T2 \rightarrow T3$.

The graph is acyclic. Therefore (a) is conflict-serializable.

A topological order consistent with the edges is:

$$\boxed{T1 \rightarrow T2 \rightarrow T3}$$

(so the schedule is conflict-equivalent to the serial schedule $T1, T2, T3$).

(b) $S_b : r_1(X); r_3(X); w_3(X); w_1(X); r_2(X)$.

Conflicts:

- $r_1(X)$ (pos 1) before $w_3(X)$ (pos 3) $\Rightarrow T1 \rightarrow T3$ (RW).
- $w_3(X)$ (pos 3) before $w_1(X)$ (pos 4) $\Rightarrow T3 \rightarrow T1$ (WW).
- $w_1(X)$ (pos 4) before $r_2(X)$ (pos 5) $\Rightarrow T1 \rightarrow T2$ (WR).
- $w_3(X)$ (pos 3) before $r_2(X)$ (pos 5) $\Rightarrow T3 \rightarrow T2$ (WR).

Precedence graph contains $T1 \rightarrow T3$ and $T3 \rightarrow T1$ (a cycle).

Therefore (b) is NOT conflict-serializable.

(c) $S_c : r_3(X); r_2(X); w_3(X); r_1(X); w_1(X)$.

Conflicts:

- $r_2(X)$ (pos 2) before $w_3(X)$ (pos 3) $\Rightarrow T2 \rightarrow T3$ (RW).
- $w_3(X)$ (pos 3) before $r_1(X)$ (pos 4) $\Rightarrow T3 \rightarrow T1$ (WR).
- $w_3(X)$ (pos 3) before $w_1(X)$ (pos 5) $\Rightarrow T3 \rightarrow T1$ (WW).
- $r_2(X)$ (pos 2) before $w_1(X)$ (pos 5) $\Rightarrow T2 \rightarrow T1$ (RW).

Precedence graph edges: $T2 \rightarrow T3$, $T3 \rightarrow T1$, $T2 \rightarrow T1$.

The graph is acyclic. Therefore (c) is conflict-serializable.

A valid topological order is:

$$\boxed{T2 \rightarrow T3 \rightarrow T1}$$

(so the schedule is conflict-equivalent to the serial schedule $T2, T3, T1$).

(d) $S_d : r_3(X); r_2(X); r_1(X); w_3(X); w_1(X)$.

Conflicts:

- $r_2(X)$ (pos 2) before $w_3(X)$ (pos 4) $\Rightarrow T2 \rightarrow T3$ (RW).
- $r_1(X)$ (pos 3) before $w_3(X)$ (pos 4) $\Rightarrow T1 \rightarrow T3$ (RW).
- $w_3(X)$ (pos 4) before $w_1(X)$ (pos 5) $\Rightarrow T3 \rightarrow T1$ (WW).
- $r_2(X)$ (pos 2) before $w_1(X)$ (pos 5) $\Rightarrow T2 \rightarrow T1$ (RW).

The graph contains $T1 \rightarrow T3$ and $T3 \rightarrow T1$ (a cycle).

Therefore (d) is NOT conflict-serializable.

Final summary:

- (a) Serializable. Equivalent serial schedule: $T1 \rightarrow T2 \rightarrow T3$.
- (b) Not serializable.
- (c) Serializable. Equivalent serial schedule: $T2 \rightarrow T3 \rightarrow T1$.
- (d) Not serializable.

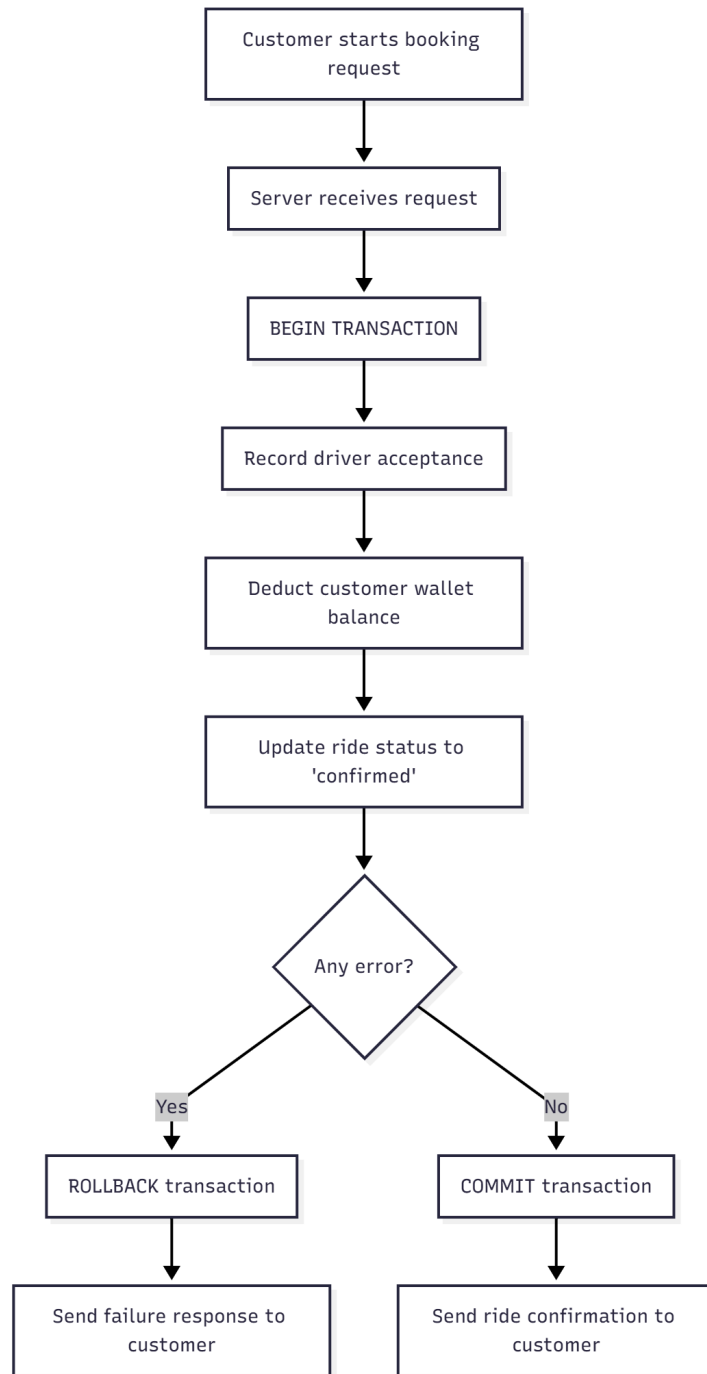


Figure 1: Flow for part c explanation

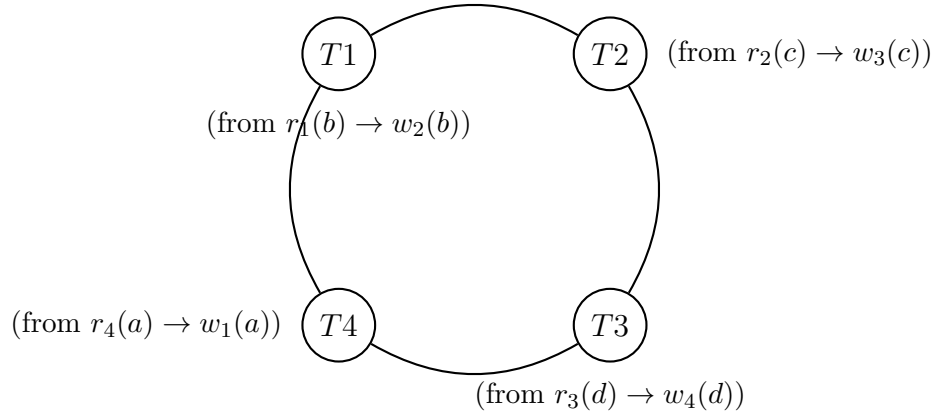


Figure 2: Precedence graph for schedule $S1$ (contains a cycle).

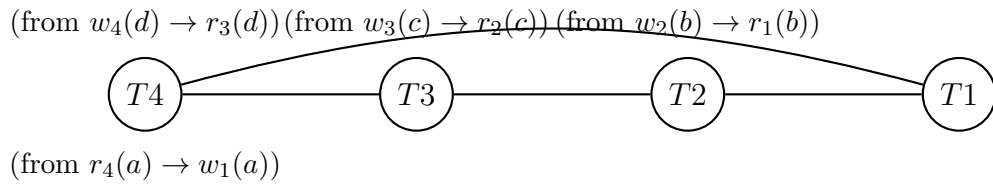


Figure 3: Precedence graph for schedule $S2$ (acyclic).